

NGINX Reverse Proxy Assignment

Short Project Report

Problem Statement

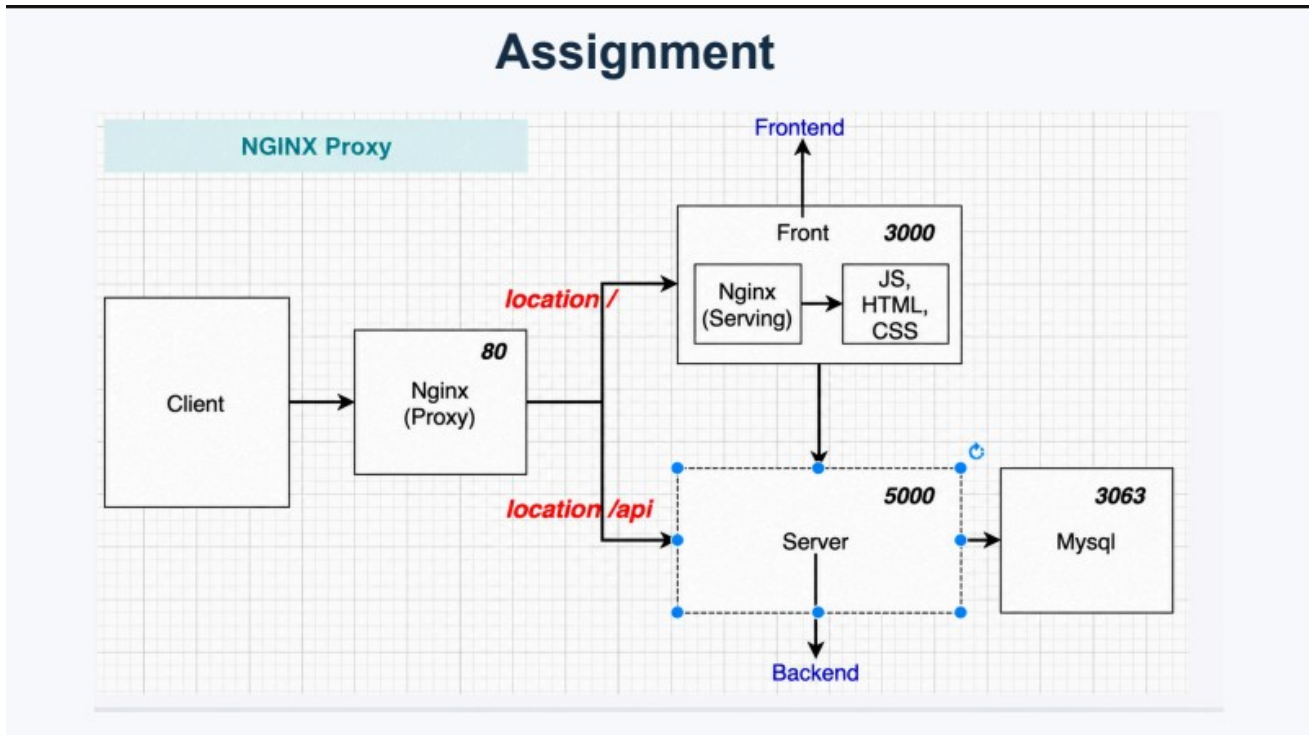


Figure 1. Assignment architecture given in the problem statement

Goal: I build a small web setup where NGINX works as a reverse proxy. It sends normal web requests to the frontend and API requests to the backend. The backend connects to MySQL and stores every API request.

GitHub Repository: <https://github.com/nkswapnil087/reverse-proxy-assignment-mcsc>

Client -> NGINX :8080 -> Frontend :3000
-> Backend :5000 -> MySQL :3306

1. Steps I Follow

1. I create separate frontend and backend services using Python.
2. I run the frontend on 127.0.0.1:3000 and the backend on 127.0.0.1:5000.
3. I set up MySQL on 127.0.0.1:3306 and create the reverse_proxy_assignment database.
4. I create a requests table so the backend can save each API request.
5. I configure NGINX to listen on port 8080.
6. I route / to the frontend and /api to the backend.
7. I test the frontend and API through NGINX.
8. I check the MySQL rows to confirm that the backend saves the requests.
9. I check the IP addresses and listening ports to confirm the network setup.

2. NGINX Configuration

I use NGINX as the single entry point of the project. It listens on port 8080. When the path starts with /api it sends the request to the backend on port 5000. For other paths it sends the request to the frontend on port 3000.

```
location /api {
    proxy_pass http://127.0.0.1:5000;
}

location / {
    proxy_pass http://127.0.0.1:3000;
}
```

```
nginx-reverse-proxy-assignment > nginx > ⚙ nginx.conf
1  worker_processes 1;
2  pid /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/nginx.pid;
3  error_log /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/logs/nginx-error.log info;
4
5  events { worker_connections 1024; }
6
7  http {
8      access_log /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/logs/nginx-access.log;
9      client_body_temp_path /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/client_body;
10     proxy_temp_path /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/proxy_temp;
11     fastcgi_temp_path /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/fastcgi_temp;
12     uwsgi_temp_path /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/uwsgi_temp;
13     scgi_temp_path /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/run/scgi_temp;
14     include /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/nginx/mime.types;
15     default_type application/octet-stream;
16     sendfile on;
17
18     server {
19         listen 8080;
20         server_name localhost;
21
22         location /api {
23             proxy_pass http://127.0.0.1:5000;
24             proxy_set_header Host $host;
25             proxy_set_header X-Real-IP $remote_addr;
26             proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
27         }
28
29         location / {
30             proxy_pass http://127.0.0.1:3000;
31             proxy_set_header Host $host;
32             proxy_set_header X-Real-IP $remote_addr;
33             proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
34         }
35     }
36 }
37
```

Figure 2. NGINX configuration used in the project

I also test the NGINX configuration before using it. The test shows that the syntax is correct and the configuration is successful.

```
nk-swapnil@nk-swapnil-L00-15IAX9E:~/Desktop/nginx-reverse-proxy-assignment$ ./local-nginx/usr/sbin/nginx \
-p "$PWD/local-nginx" \
-c "$PWD/nginx/nginx.conf" \
-t
nginx: the configuration file /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/nginx/nginx.conf syntax is ok
nginx: configuration file /home/nk-swapnil/Desktop/nginx-reverse-proxy-assignment/nginx/nginx.conf test is successful
```

Figure 3. NGINX configuration validation

3. IP Address Explanation

I use 127.0.0.1 for the frontend backend and MySQL. This is the loopback address. It means these services are available only from my own computer. NGINX listens on 0.0.0.0:8080. This means NGINX can accept requests through the available network interfaces.

My wired Ethernet interface enp7s0 has the private IP address 192.168.0.107/24. My Wi-Fi interface wlp0s20f3 has 192.168.0.102/24. These are private LAN addresses.

```
nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/nginx-reverse-proxy-assignment$ ip -br addr
lo                UNKNOWN          127.0.0.1/8  ::1/128
enp7s0            UP                192.168.0.107/24  fe80::51c4:2779:a441:ab1/64
wlp0s20f3         UP                192.168.0.102/24  fe80::24da:a040:800c:cb21/64
docker0           DOWN              172.17.0.1/16
nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/nginx-reverse-proxy-assignment$
```

Figure 4. Network interfaces and IP addresses

The listening port check shows that NGINX is open on 0.0.0.0:8080. The frontend backend and MySQL stay on 127.0.0.1. This keeps the internal services behind NGINX.

```
nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/nginx-reverse-proxy-assignment$ ss -ltnp | grep -E '8080|3000|5000|3306'
LISTEN 0      511          0.0.0.0:*      0.0.0.0:*      users: (("nginx",pid=32306,fd=5),("nginx",pid=32305,fd=5))
LISTEN 0       5           127.0.0.1:5000  0.0.0.0:*      users: (("python3",pid=32300,fd=3))
LISTEN 0       5           127.0.0.1:3000  0.0.0.0:*      users: (("python3",pid=32301,fd=3))
LISTEN 0      151          127.0.0.1:3306  0.0.0.0:*      users: (("mysqld",pid=32227,fd=20))
nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/nginx-reverse-proxy-assignment$
```

Figure 5. Listening ports for NGINX frontend backend and MySQL

4. Request Flow

When I open <http://127.0.0.1:8080/> the browser sends the request to NGINX. NGINX matches the / location and forwards the request to 127.0.0.1:3000. The frontend then returns the HTML CSS and JavaScript.

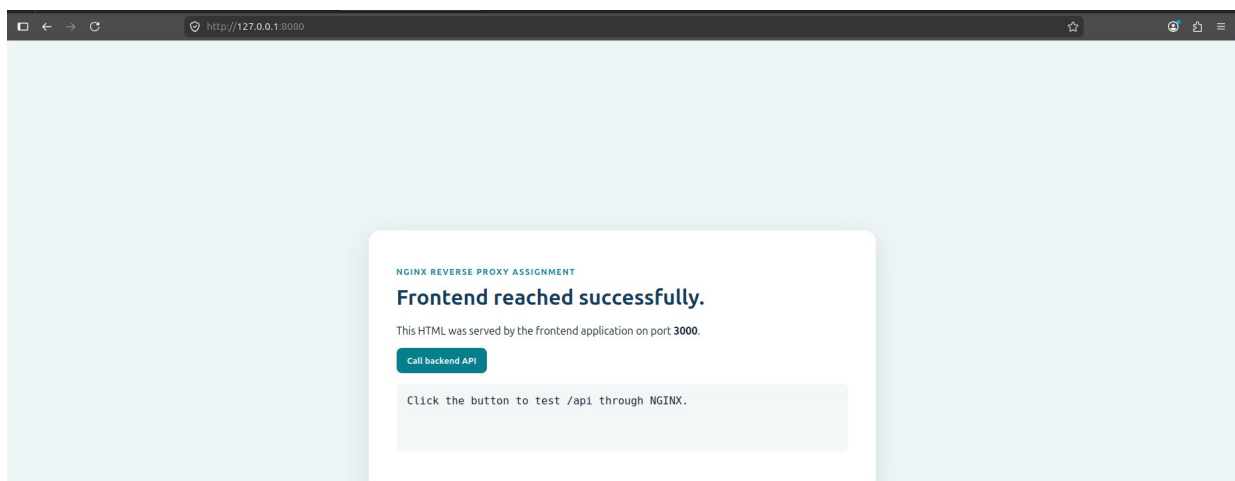


Figure 6. Frontend reached through NGINX on port 8080

When I open `http://127.0.0.1:8080/api` NGINX matches the `/api` location. It forwards the request to the backend on `127.0.0.1:5000`. The backend records the request in MySQL. It then returns a JSON response through NGINX to the browser.

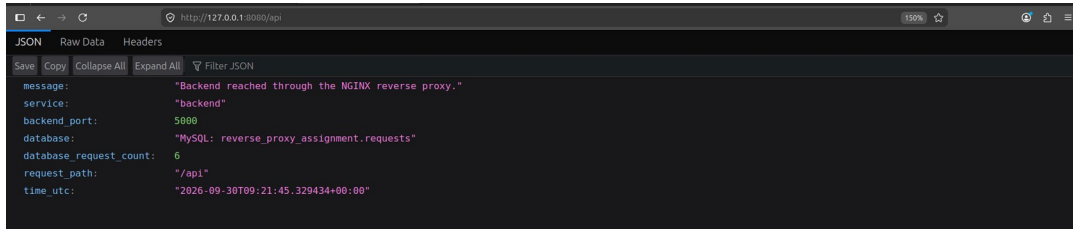


Figure 7. Backend API reached through NGINX

The MySQL table shows the saved `/api` requests. This proves that the backend connects to the database and stores data correctly.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_reverse_proxy_assignment |
+-----+
| requests                            |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM requests;
+----+-----+-----+
| id | path | created_at |
+----+-----+-----+
| 1  | /api | 2026-09-30 15:01:26.312001 |
| 2  | /api | 2026-09-30 15:01:26.323413 |
| 3  | /api | 2026-09-30 15:01:34.997996 |
| 4  | /api | 2026-09-30 15:02:18.388585 |
| 5  | /api/ | 2026-09-30 15:02:49.806507 |
| 6  | /api | 2026-09-30 15:21:45.304299 |
+----+-----+-----+
6 rows in set (0.00 sec)

mysql> █
```

Figure 8. API requests saved in the MySQL requests table

5. Result

I successfully make NGINX work as a reverse proxy. I use one entry point on port 8080. NGINX sends frontend requests to port 3000 and API requests to port 5000. The backend connects to MySQL on port 3306 and stores the API requests. The screenshots show that the full request flow works correctly.